

4. Spark ML 소개와 ML 모델 빌딩

Spark ML 소개

Spark ML 실습: ML 모델 만들어보기

Contents

1. Spark ML 소개
2. 실습: 머신러닝 모델 만들기
3. Spark ML 피쳐변환
4. Spark ML Pipeline 살펴보기



Spark ML 소개

Spark이 제공하는 머신러닝에 대해 배워보자

◆ Spark ML 소개 (1)

- ❖ 머신러닝 관련 다양한 알고리즘, 유틸리티로 구성된 라이브러리
 - Classification, Regression, Clustering, Collaborative Filtering, Dimensionality Reduction. 전체 리스트는 [링크](#) 참고
 - 아직 딥러닝은 지원은 아직 미약
- ❖ 여기에는 **RDD** 기반과 데이터프레임 기반의 두 버전이 존재
 - spark.mllib vs. spark.ml
 - spark.mllib가 RDD 기반이고 spark.ml은 데이터프레임 기반
 - spark.mllib는 RDD위에서 동작하는 이전 라이브러리로 더 이상 업데이트가 안됨
 - 항상 spark.ml을 사용할 것!
 - import pyspark.ml (~~import pyspark.mllib~~)

◆ Spark ML의 장점

❖ 원스톱 ML 프레임워크!

- 데이터프레임과 SparkSQL등을 이용해 전처리
- Spark MLlib를 이용해 모델 빌딩
- ML Pipeline을 통해 모델 빌딩 자동화
- MLflow로 모델 관리하고 서빙

❖ 대용량 데이터도 처리 가능!

◆ Spark ML 소개: MLflow

❖ 모델의 관리와 서빙을 위한 Ops 관련 기능도 제공

❖ MLflow

- 모델 개발과 테스트와 관리와 서빙까지 제공해주는 End-to-End 프레임워크
- MLflow는 파이썬, 자바, R, API를 지원
- MLflow는 트래킹(Tracking), 모델(Models), 프로젝트(Projects)를 지원

◆ Spark ML 제공 알고리즘

❖ Classification:

- Logistic regression, Decision tree, Random forest, Gradient-boosted tree, ...

❖ Regression:

- Linear regression, Decision tree, Random forest, Gradient-boosted tree, ...

❖ Clustering:

- K-means, LDA(Latent Dirichlet Allocation), GMM(Gaussian Mixture Model), ..

❖ Collaborative Filtering

- 명시적인 피드백과 암묵적인 피드백 기반
- 명시적인 피드백의 예) 리뷰 평점
- 암묵적인 피드백의 예) 클릭, 구매 등등

실습: 머신러닝 모델 만들기

보스턴 주택가격 예측 모델 만들기: Regression

타이타닉 승객 생존 예측 모델 만들기: Classification

◆ Spark ML 기반 모델 빌딩의 기본 구조

❖ 여느 라이브러리를 사용한 모델 빌딩과 크게 다르지 않음

- 트레이닝셋 전처리
- 모델 빌딩
- 모델 검증 (confusion matrix)

❖ Scikit-Learn과 비교했을 때 장점

- 차이점은 결국 데이터의 크기
 - Scikit-Learn은 하나의 컴퓨터에서 돌아가는 모델 빌딩
 - Spark MLlib는 여러 서버 위에서 모델 빌딩
- 트레이닝셋의 크기가 크면 전처리와 모델 빌딩에 있어 **Spark**이 큰 장점을 가짐
- Spark은 ML 파이프라인을 통해 모델 개발의 반복을 쉽게 해줌

◆ 보스턴 주택가격 예측

- ❖ 1970년대 미국 인구조사 서비스 (US Census Service)에서 보스턴 지역의 주택 가격 데이터를 수집한 데이터를 기반으로 모델 빌딩
- ❖ 트레이닝셋은 [여기](#)서 다운로드 (원래 데이터는 [CMU Stat Lib Archive](#))
 - 개별 주택가격의 예측이 아니라 지역별 중간 주택가격 예측임
- ❖ Regression 알고리즘 사용 예정
 - 연속적인 주택가격을 예측이기에 Classification 알고리즘은 사용불가

- ◆ 보스턴 주택가격 트레이닝 셋 보기
 - ❖ 총 506개의 레코드로 구성되며 13개의 피쳐와 레이블 필드(주택가격)로 구성
 - 506개 동네의 주택 중간값 데이터임 (개별 주택이 아님에 유의)
 - 14번째 필드가 바로 예측해야하는 중간 주택 가격

필드 이름	설명
1. CRIM	주택이 있는 지역의 인당 범죄율
2. ZN	25000 sqft (대략 700평) 이상의 땅이 주거지역으로 설정된 비율
3. INDUS	에이커당 공업단지의 비율
4. CHAS	주택이 강가에 위치한 비율
5. NOX	산화질소 농도로 오염정도를 나타냄
6. RM	주택당 평균 방의 수
7. AGE	1940년 전에 지어진 주택의 비율

필드 이름	설명
8. DIS	보스턴 지역 고용 센터까지의 평균 거리
9. RAD	고속도로 접근성에 대한 인덱스
10. TAX	재산세 (주택가격 \$10K 기준)
11. PTRATO	초등학교 학생-선생님의 비율
12. B	흑인 인구의 비율
13. LSTAT	저소득자의 인구 비율
14. MEDV	천불 단위의 주택 평균값

◆ 타이타닉 승객 생존 예측

- ❖ 머신러닝의 Hello World라고 할 수 있는 굉장히 유명한 데이터셋
 - 2015년 캐글에서 “Titanic - Machine Learning from Disaster”라는 이름의 튜토리얼로 시작됨
 - <https://www.kaggle.com/c/titanic>
- ❖ Binary Classification 알고리즘 사용 예정
 - 생존 혹은 비생존을 예측하는 것이라 Binary Classification을 사용
 - 정확히는 Binomial Logistic Regression을 사용 (2개 클래스 분류기)
 - AUC (Area Under the Curve)의 값이 중요한 성능 지표가 됨
 - True Positive Rate과 False Positive Rate
 - True Positive Rate: 생존한 경우를 얼마나 맞게 예측했나? 흔히 Recall이라고 부르기도 함
 - False Positive Rate: 생존하지 못한 경우를 생존한다고 얼마나 예측했나?

◆ 타이타닉 승객 생존 예측

❖ 총 892개의 레코드로 구성되며 11개의 피쳐와 레이블 필드(생존여부)로 구성

- 2번째 필드(Survived) 바로 예측해야하는 승객 생존 여부

필드 이름	설명
1. PassengerId	승객에게 주어진 일련번호
2. Survived	생존여부를 나타내는 레이블 정보
3. Pclass	티켓클래스. 1 = 1st, 2 = 2nd, 3 = 3rd
4. Name	승객의 이름
5. Gender	승객의 성별
6. Age	승객의 나이

필드 이름	설명
7. SibSp	같이 승선한 형제/자매와 배우자의 수
8. Parch	같이 승선한 부모와 자녀의 수
9. Ticket	티켓 번호
10. Fare	운임의 값
11. Cabin	숙소 번호
12. Embarked	승선한 항구. C = Cherbourg, Q = Queenstown, S = Southampton



Spark ML 피쳐 변환

Spark ML가 제공하는 피쳐변환에 대해 배워보자

◆ 피쳐 추출과 변환

- ❖ 피쳐 값들을 모델 훈련에 적합한 형태로 바꾸는 것을 지칭
- ❖ 크게 두 가지가 존재: **Feature Extractor**와 **Feature Transformer**
- ❖ **Feature Transformer**가 하는 일
 - 먼저 피쳐 값들은 숫자 필드이어야함
 - 텍스트 필드(카테고리 값들)를 숫자 필드로 변환해야함
 - 숫자 필드 값의 범위 표준화
 - 숫자 필드라고 해도 가능한 값의 범위를 특정 범위(0부터 1)로 변환해야 함
 - 이를 피쳐 스케일링 (**Feature Scaling**) 혹은 정규화 (**Normalization**)
 - 비어있는 필드들의 값을 어떻게 채울 것인가?
 - **Imputer**. 앞서 타이타닉 승객 생존 분류기에 써봤음

◆ 피쳐 추출과 변환

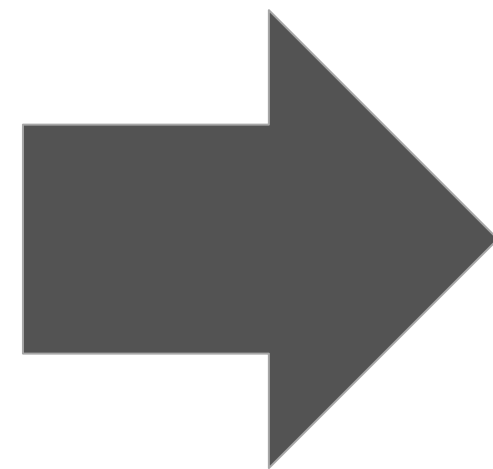
❖ Feature Extractor가 하는 일

- 기존 피쳐에서 새로운 피쳐를 추출
- TF-IDF, Word2Vec, ...
 - 많은 경우 텍스트 데이터를 어떤 형태로 인코딩하는 것이 여기에 해당함

◆ 피쳐 변환 **StringIndexer**: 텍스트 카테고리를 숫자로 변환 (1)

- ❖ 아래 왼쪽과 같은 값을 갖는 **Color**라는 이름의 피쳐가 존재한다면
- ❖ 이를 오른쪽과 같은 숫자로 변환해주는 것이 피쳐변환의 목적

변환 전
Red
Blue
Orange
White
Black
Gray
Yellow
Pink
Green
Purple



변환 후
1
2
3
4
5
6
7
8
9
10

- ◆ 피쳐 변환 **StringIndexer**: 텍스트 카테고리를 숫자로 변환 (2)
 - ❖ Scikit-Learn은 `sklearn.preprocessing` 모듈 아래 여러 인코더 존재
 - OneHotEncoder, LabelEncoder, OrdinalEncoder, ...
 - ❖ Spark ML의 경우 `pyspark.ml.feature` 모듈 밑에 두 개의 인코더 존재
 - StringIndexer, OneHotEncoder
 - 사용법은 Indexer 모델을 만들고(fit), Indexer 모델로 데이터프레임을 transform

```
from pyspark.ml.feature import StringIndexer
```

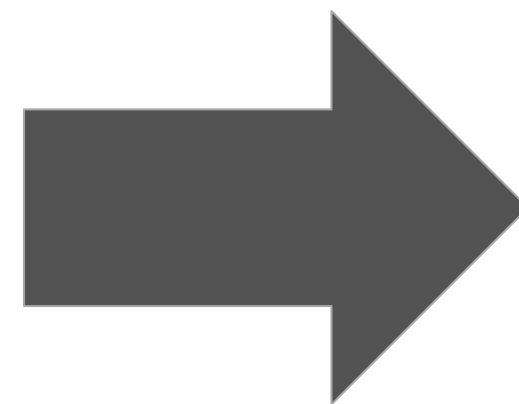
```
gender_indexer = StringIndexer(inputCol='Gender', outputCol='GenderIndexed')
```

```
gender_indexer_model = gender_indexer.fit(final_data)
```

```
final_data_with_transformed_gender_gender = gender_indexer_model.transform(final_data)
```

- ◆ 피쳐 변환 **Scaler**: 숫자 필드값의 범위 표준화 (1)
 - ❖ 숫자 필드 값의 범위를 특정 범위(예를 들면 0부터 1)로 변환하는 것
 - ❖ 피쳐 스케일링 (**Feature Scaling**) 혹은 정규화 (**Normalization**)라 부름

변환 전
-20
100
40
25
15

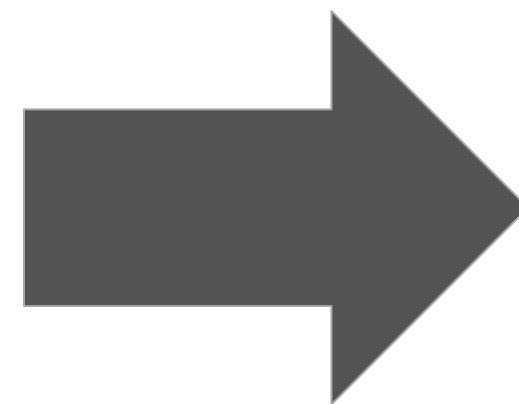


변환 후
0
1
0.5
0.375
0.125

- ◆ 피쳐 변환 **Scaler**: 숫자 필드값의 범위 표준화 (2)
 - ❖ Scikit-Learn은 `sklearn.preprocessing` 모듈 아래 두 개의 스케일러 존재
 - `StandardScaler`, `MinMaxScaler`
 - ❖ Spark ML의 경우 `pyspark.ml.feature` 모듈 밑에 두 개의 스케일러 존재
 - `StandardScaler`, `MinMaxScaler`
 - 사용법은 **Scaler** 모델을 만들고(`fit`), **Scaler** 모델로 데이터프레임을 **transform**
 - ❖ **StandardScaler**
 - 각 값에서 평균을 빼고 이를 표준편차로 나눔. 값의 분포가 정규분포를 따르는 경우 사용
 - ❖ **MinMaxScaler**
 - 모든 값을 0과 1사이로 스케일. 각 값에서 최소값을 빼고 (최대값-최소값)으로 나눔

- ◆ 피쳐 변환 **Imputer**: 값이 없는 필드 채우기 (1)
 - ❖ 값이 존재하지 않는 레코드들이 존재하는 필드들의 경우 기본값을 정해서 채우는 것. **Impute**한다고 부름

변환 전
10
20
30
40



변환 후
10
25
20
30
40

- ◆ 피쳐 변환 **Imputer**: 값이 없는 필드 채우기 (2)
 - ❖ Scikit-Learn은 `sklearn.preprocessing` 모듈 아래 존재
 - **Imputer**
 - ❖ Spark ML의 경우 `pyspark.ml.feature` 모듈 밑에 존재
 - **Imputer**
 - 사용법은 **Imputer** 모델을 만들고(`fit`), **Imputer** 모델로 데이터프레임을 `transform`

```
from pyspark.ml.feature import Imputer
```

```
imputer = Imputer(strategy='mean', inputCols=['Age'], outputCols=['AgeImputed'])
```

```
imputer_model = imputer.fit(final_data)
```

```
final_data_age_transformed = imputer_model.transform(final_data)
```




Spark ML Pipeline 살펴보기

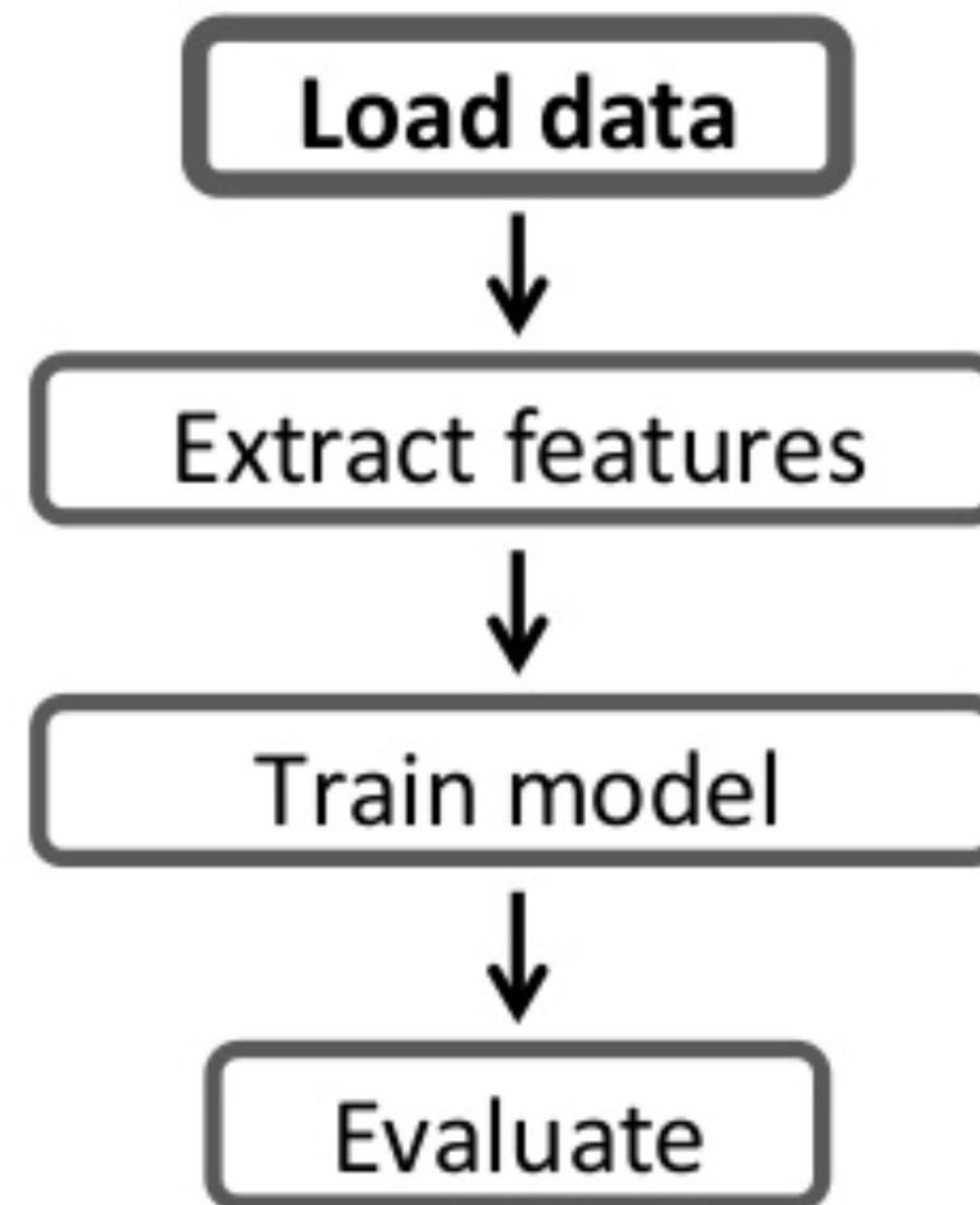
모델 빌딩과 테스트 과정을 자동화하자!

◆ 모델 빌딩과 관련된 흔한 문제들

- a. 트레이닝 셋의 관리가 안됨
- b. 모델 훈련 방법이 기록이 안됨
 - 어떤 트레이닝 셋을 사용했는지?
 - 어떤 피쳐들을 사용했는지?
 - 하이퍼 파라미터는 무엇을 사용했는지?
- c. 모델 훈련에 많은 시간 소요
 - 모델 훈련이 자동화가 안된 경우 매번 각 스텝들을 노트북 등에서 일일이 수행
 - 에러가 발생할 여지가 많음 (특정 스텝을 까먹거나 조금 다른 방식 적용)

◆ ML Pipeline의 등장

- ❖ 앞서 언급한 문제들 중 b와 c를 해결!
- ❖ 자동화를 통해 에러 소지를 줄이고 반복을 빠르게 가능하게 해줌



◆ Spark ML 관련 개념 정리

❖ ML 파이프라인이란?

- 데이터 과학자가 머신러닝 개발과 테스트를 쉽게 해주는 기능 (데이터 프레임 기반)
- 머신러닝 알고리즘에 관계없이 일관된 형태의 **API**를 사용하여 모델링이 가능
- ML 모델개발과 테스트를 반복가능해줌
 - 뒤에 설명할 **Transformer**와 **Estimator**로 구성됨.

❖ 4개의 요소로 구성

- 데이터프레임
- Transformer
- Estimator
- Parameter

◆ Spark ML 관련 개념 정리

❖ ML 파이프라인의 구성요소: 데이터 프레임

- ML 파이프라인에서는 데이터프레임이 기본 데이터 포맷
- 기본적으로 **CSV, JSON, Parquet, JDBC**(관계형 데이터베이스)를 지원
- ML 파이프라인에서 다음 2가지의 새로운 데이터소스를 지원
 - 이미지 데이터소스
 - jpeg, png 등의 이미지들을 지정된 디렉토리에서 로드
 - LIBSVM 데이터소스
 - label과 features 두 개의 컬럼으로 구성되는 머신러닝 트레이닝셋 포맷
 - features 컬럼은 벡터 형태의 구조를 가짐

◆ Spark ML 관련 개념 정리

❖ ML 파이프라인의 구성요소: **Transformer (1)**

- 입력 데이터프레임을 다른 데이터프레임으로 변환
 - 하나 이상의 새로운 컬럼을 추가
- 2 종류의 Transformer가 존재하며 **transform**이 메인 함수
 - Feature Transformer와 Learning Model

◆ Spark ML 관련 개념 정리

❖ ML 파이프라인의 구성요소: **Transformer (2)**

● Feature Transformer

- 입력 데이터프레임의 컬럼으로부터 새로운 컬럼을 만들어내 이를 추가한 새로운 데이터프레임을 출력으로 내줌. 보통 피쳐 엔지니어링을 하는데 사용
- 예) Imputer, StringIndexer, VectorAssembler
 - Imputer는 기본값 지정에 사용
 - StringIndexer는 categorical 정보를 숫자 정보로 변환
 - VectorAssembler: 주어진 컬럼들을 통합하여 하나의 벡터 컬럼으로 변환

◆ Spark ML 관련 개념 정리

❖ ML 파이프라인의 구성요소: **Transformer (3)**

- Learning Model

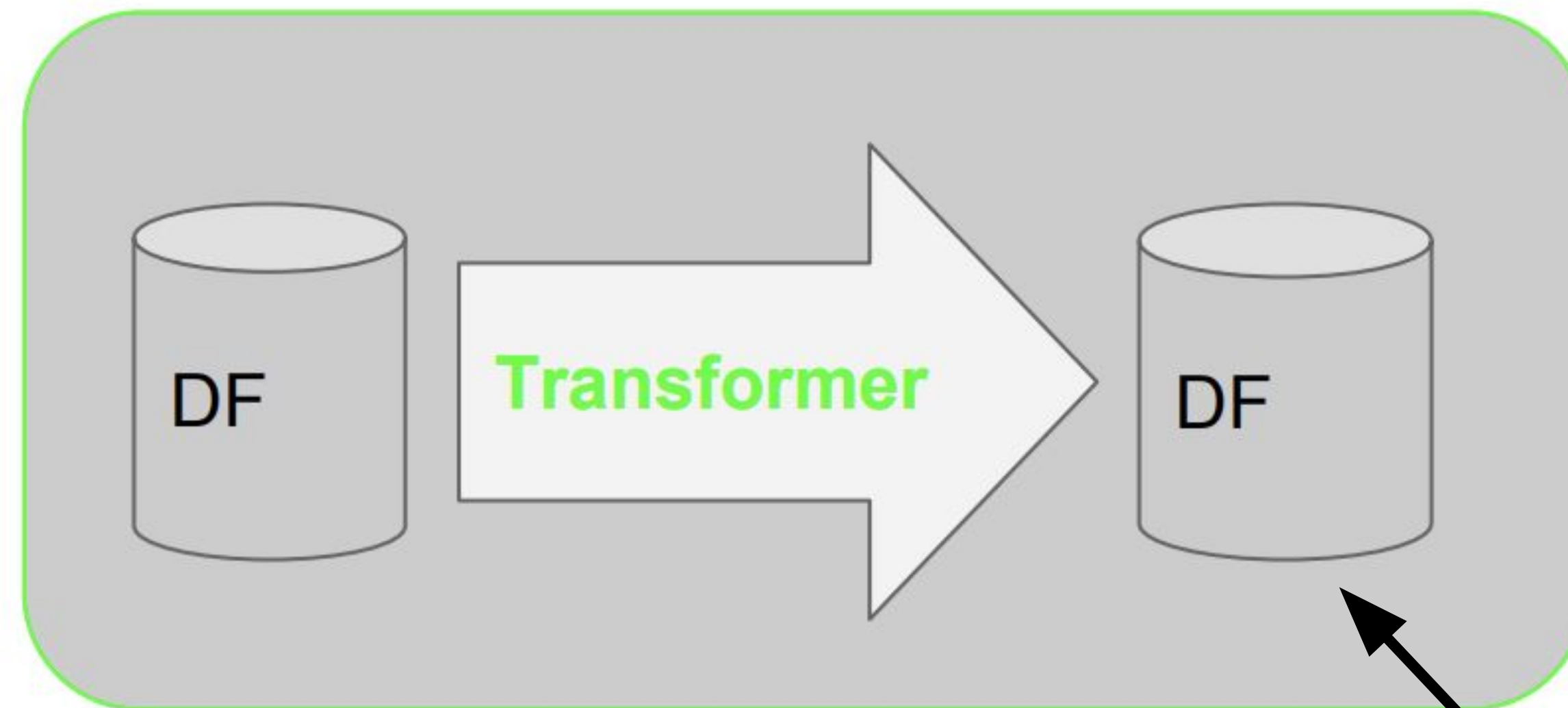
- 머신러닝 모델에 해당.
- 피쳐 데이터프레임을 입력으로 받아 예측값이 새로운 컬럼으로 포함된 데이터프레임을 출력으로 내줌: prediction, probability

◆ Spark ML 관련 개념 정리

❖ ML 파이프라인의 구성요소: **Feature Transformer**

- transform 함수

f1	f2	f3
1	2	3
2	3	4
3	4	5



f1	f2	f3	f4
1	2	3	4
2	3	4	5
3	4	5	6

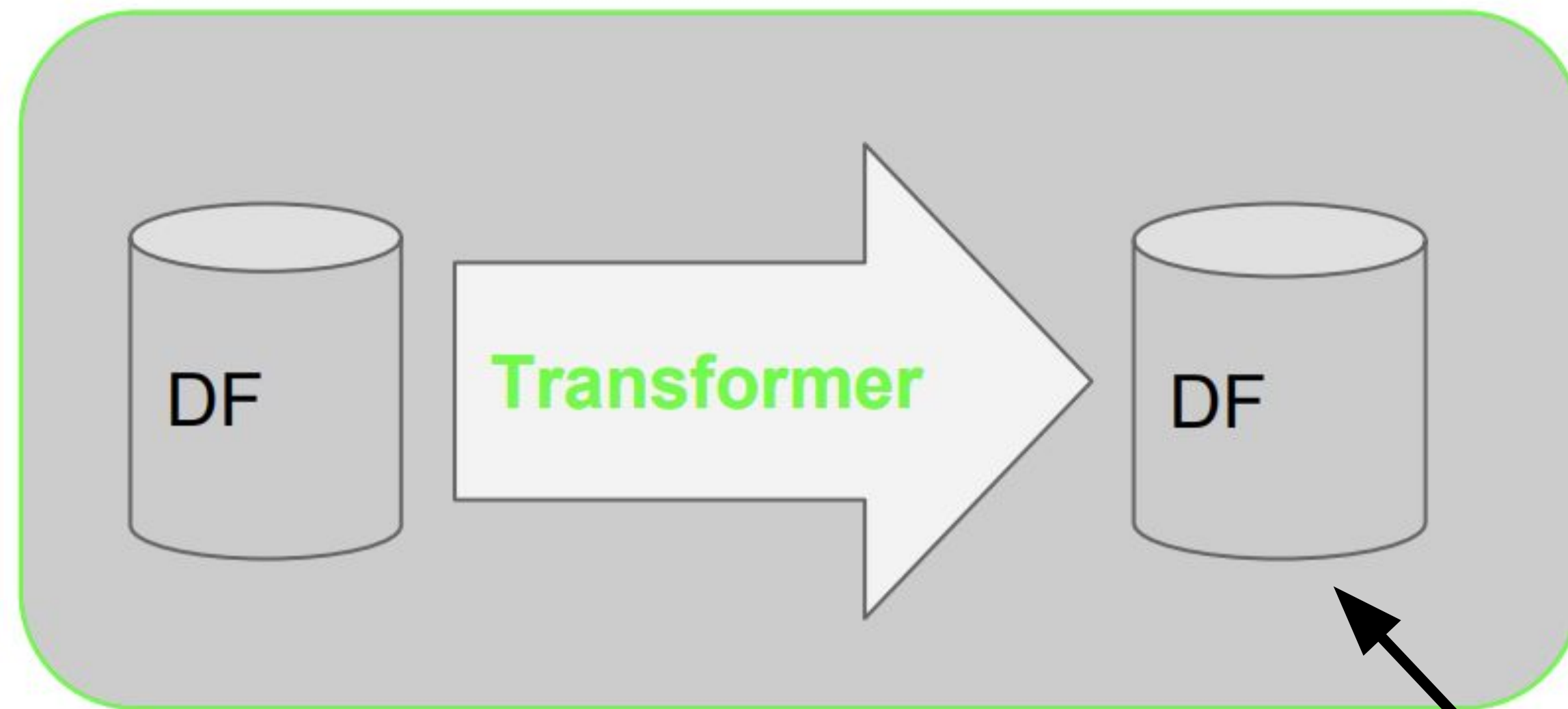
하나 이상의 컬럼이
추가됨

◆ Spark ML 관련 개념 정리

❖ ML 파이프라인의 구성요소: **Learning Model**

- 머신러닝 모델에 해당. 역시 transform 함수로 동작

f1	f2	f3
1	2	3
2	3	4
3	4	5



f1	f2	f3	prediction
1	2	3	4
2	3	4	5
3	4	5	6

prediction 컬럼이 추가됨

◆ Spark ML 관련 개념 정리

❖ ML 파이프라인의 구성요소: **Estimator (1)**

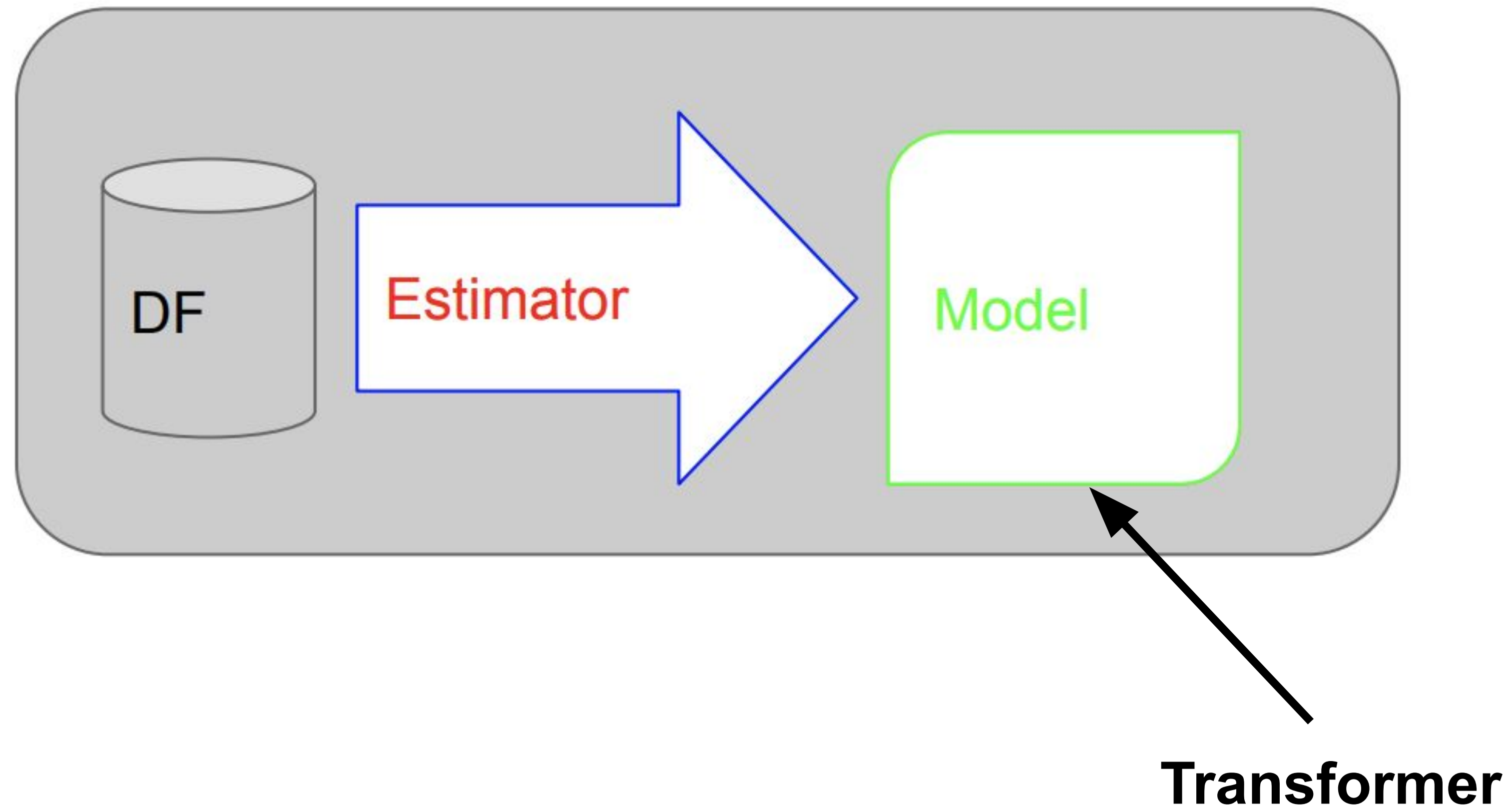
- 머신러닝 알고리즘에 해당. **fit**이 메인 함수
 - 트레이닝셋 데이터프레임을 입력으로 받아서 머신러닝 모델(Transformer)을 만들어냄
 - 입력: 데이터프레임 (트레이닝 셋)
 - 출력: 머신러닝 모델
 - 예를 들어 LogisticRegression은 Estimator이고 LogisticRegression.fit()를 호출하면 머신 러닝 모델(Transformer)을 만들어냄

◆ Spark ML 관련 개념 정리

❖ ML 파이프라인의 구성요소: **Estimator (2)**

- ML Pipeline도 Estimator
- Estimator는 저장과 읽기 함수 제공
 - 즉 모델과 ML Pipeline을 저장했다가 나중에 다시 읽을 수 있음
 - save와 load

- ◆ Spark ML 관련 개념 정리
 - ❖ ML 파이프라인의 구성요소: **Estimator**



◆ Spark ML 관련 개념 정리

❖ ML 파이프라인의 구성요소: **Parameter**

- Transformer와 Estimator의 공통 API로 다양한 인자를 적용해줌
- 두 종류의 파라미터가 존재:
 - Param(하나의 이름과 값)
 - ParamMap (Param 리스트)
- 파라미터의 예)
 - 훈련 반복수 (iteration) 지정을 위해 setMaxIter()를 사용
 - ParamMap(lr.maxIter -> 10)
- 파라미터는 fit (Estimator) 혹은 transform (Transformer)에 인자로 지정 가능

◆ Spark ML 관련 개념 정리

◆ ML Pipeline

- 하나 이상의 Transformer와 Estimator가 연결된 모델링 워크플로우
 - 입력은 데이터프레임
 - 출력은 머신러닝 모델
- ML Pipeline 그 자체도 Estimator
 - 따라서 ML Pipeline의 실행은 fit 함수의 호출로 시작
 - 저장했다가 나중에 다시 로딩하는 것이 가능 (Persistence)
- 한번 파이프라인을 만들면 반복 모델빌딩이 쉬워짐

요약

Spark ML와 제공 기능에 대해 배움

pyspark.ml을 이용해 실제 머신러닝 모델을 만들어봄

ML Pipeline을 비롯한 다양한 Spark ML 기능에 대해 배움



Grepp, Inc

한기용

keeyonghan@hotmail.com